← Back to Articles

What is test-time compute and how to scale it?

Community Article Published February 6, 2025


Ksenia Se
Kseniase Follow


Alyona Vert
alyona0l Follow

we dive into test-time compute and discuss five+ open-source methods for its effective scaling for deep step-by-step models' reasoning.

For a long time, many AI and ML researchers and users preferred models that generate outputs immediately. But the recent shift to slow thinking, introduced with OpenAI's o1 model, turned everything upside down. Since this breakthrough, it has become clear how remarkable a model's reasoning capabilities can be when it is not "in a hurry" and has time to "think" through

Papers mentioned in this article 10

o1-Coder: an o1 Replication for Coding
Paper • 2412.00154 • Published Nov 29, 2... • △ 44

OpenAI o1 System Card
Paper • 2412.16720 • Published Dec 21, 2... • △ 38

Mulberry: Empowering MLLM with o1-like Rea...
Paper • 2412.18319 • Published Dec 24, 2... • △ 39

Virgo: A Preliminary Exploration on Reproduci...
Paper • 2501.01904 • Published Jan 3, 2025 • △ 33

Test-time Computing: from System-1 Thinking t...
Paper • 2501.02497 • Published Jan 5, 2025 • △ 45

multiple steps – a process known as Chain-of-Thought reasoning. These aspects tie into a fascinating topic: test-time compute. Today, we'll take a broader look at test-time compute, discussing five methods to scale it and how it can enhance AI models' reasoning. This article is a true gem!

DeepSeek-R1: Incentivizing Reasoning Capabilit...
Paper • 2501.12948 • Published Jan 22, 2... • △ 451

📩 **Click follow! If you want to receive our articles straight to your inbox, please** subscribe here

---

In today's episode, we will cover:

- The core idea behind OpenAI's o1 model
- What basically is Test-Time Compute (TTC)?
- DeepSeek-R1's way of scaling test-time compute
- Test-time compute scaling meets multimodality
    - What if we use long-form text examples for training MLLMs?
    - Collective learning with Collective Monte Carlo Tree Search

The core idea behind OpenAI’s o1 model

While many developers were running after high speed of input processing which leads to immediate outputs, OpenAI decided to bet on the deeper “thinking” of their o1 model, which resulted in **increasing test-time compute**. The concept of test-time compute aligns with what's known now as "System-2 thinking," which involves slow, deliberate, and logical reasoning, as opposed to "System-1 thinking," which is fast and intuitive.

What basically is Test-Time Compute (TTC)?

TTC refers to the amount of computational power used by an AI model when it is generating a response or performing a task after it has been trained. In simple terms, it's the processing power and time required when the model is actually being used, rather than when it is being trained.

Key aspects of Test-Time Compute (TTC):

Inference process: When you input a question or a prompt into a model, it processes the input and generates a response. The computational cost of this process is called test-time compute. **Scaling at test time:** Some advanced AI models, like OpenAI's o1 series, dynamically increase their reasoning time during inference. This means they can spend more time thinking about complex questions, improving accuracy at the cost of higher compute usage.

By allocating more computational resources during inference, o1 models can perform deeper reasoning, leading to more accurate and thoughtful responses. o1 uses step-by-step thinking, in other words, Chain-of-Thought method, before arriving at a final answer. Thanks to this, the o1 model excels at tasks that require complex problem-solving.

Since o1 is so powerful but, at the same time, a closed model, it has pushed other developers to create new models, based on o1 principles, trying to scale at TTC, or uncover secrets of o1 and bring these technologies to the

community. Let's dive into five research which explore, use and expand the o1's core idea to make it accessible for the developers. →

DeepSeek-R1's way of scaling test-time compute

Almost everyone is buzzing about the amazing performance results of the DeepSeek-R1 model and the power of reinforcement learning (RL), but the core idea behind adding RL remains somewhat overlooked. On the surface, R1 seems designed to compete with OpenAI's o1 model and other top models. However, as DeepSeek stated in their DeepSeek-R1 paper, the main goal was to achieve strong reasoning capabilities by leveraging the principle of deep, step-by-step thinking. The drive to develop something outstanding and the challenge of improving reasoning during inference pushed DeepSeek to explore their own approach to scaling test-time compute.

How RL and supervised fine-tuning (SFT) contributed to advanced reasoning?

DeepSeek explored **three key areas**:

- **DeepSeek-R1-Zero:** A model trained only with RL, without using any pre-labeled data.

- **DeepSeek-R1:** A model that starts with some fine-tuning on a small set of step-by-step reasoning examples before applying RL.
- **Distillation:** Transferring reasoning skills from DeepSeek-R1 to smaller AI models to make them more efficient.

Let’s look at these areas one by one:

1. **DeepSeek-R1-Zero**

DeepSeek-R1-Zero learns reasoning through pure RL using Group Relative Policy Optimization (GRPO), an adaptation of Proximal Policy Optimization (PPO). GRPO reduces training costs and improves performance by eliminating the need for a separate value function model, evaluating actions by comparing them within groups using average rewards as a baseline.

The model is rewarded for accuracy, like solving math problems with step-by-step reasoning, and for presenting its reasoning in a structured format.

It resulted in the following:

- Remarkable improvements on reasoning benchmarks like the AIME test, boosting **accuracy from 15.6% to 71%, and up to 86.7% with majority voting** – reaching OpenAI-o1 levels of performance.

- **By combining multiple answers, it reached 86.7%**, outperforming OpenAI's o1-0912 model.
- **"Aha Moment"**: With increased test-time compute, the model exhibits sophisticated behaviors such as reflection, where it revisits and reevaluates its previous steps, and the exploration of alternative problem-solving approaches. This fascinating "Aha moment" shows that the model can figure out on its own that rethinking its approach leads to better answers.
- **Self-evolution:** Over time, these behaviors emerge naturally as the model spends more time thinking through complex problems and allocates more computational resources during inference. This improves its reasoning without being explicitly programmed.

Image Credit: The original DeepSeek-R1 paper

However, DeepSeek-R1-Zero sometimes mixed languages or produced disorganized answers, making it hard to understand. This was solved through a multi-step training process of DeepSeek-R1.

2. **DeepSeek-R1 and cold-start fine-tuning with RL**

- DeepSeek-R1 starts with a small set of well-structured reasoning examples (cold-start data) before applying reasoning-focused RL. It was fine-tuned using a mix of reasoning and general-purpose tasks.
- Further it is refined using additional RL techniques to make it more helpful and aligned with human preferences in various scenarios like writing, factual QA, and self-awareness.

After these steps, DeepSeek-R1 became far more readable and user-friendly and achieved reasoning performance on par with advanced OpenAI models, like o1-1217.

Image Credit: The original DeepSeek-R1 paper

3. **Distillation of test-time compute concept**

The DeepSeek researchers transferred the reasoning capabilities of DeepSeek-R1 into smaller models to make powerful reasoning models more accessible.

They fine-tuned open-source models like Qwen and Llama using the 800,000 high-quality training examples from DeepSeek-R1. Surprisingly, unlike

DeepSeek-R1, these distilled models performed even better than applying RL directly to the smaller models:

- The distilled 7B model (DeepSeek-R1-Distill-Qwen-7B) outperformed much larger models like QwQ-32B on reasoning benchmarks.
- The distilled 32B and 70B models set new records for open-source AI reasoning tasks.

Image Credit: The original DeepSeek-R1 paper

Thanks to DeepSeek's breakthrough, we now have both large and small open-source models that introduce their own methods for effective step-by-step thinking, performing on par with or even better than OpenAI's models.

Test-time compute scaling meets multimodality

In the rising era of multimodality, researchers are also trying to apply a slow, step-by-step thinking approach to MLLMs, improving their reasoning capabilities. Here, we discuss three very different but very interesting **ways to increase test-time compute across different modalities**, including: **1)**

Fine-tuning the model using only long-form text reasoning examples; 2) Collective Monte Carlo Tree Search; 3) New test-time verification models.

What if we use long-form text examples for training MLLMs?

Since multimodal models are built on top of language models, researchers from Gaoling School of Artificial Intelligence, Baichuan AI and BAAI suggested that the slow-thinking ability of multimodal models mainly comes from their language processing component. This means the skill can be transferred across different types of data (text, images, etc.).

Indeed, their research on reproducing o1-like MLLM demonstrated that **fine-tuning the model using just 5,000 long-form reasoning examples, all text-based, led to strong results, often matching or even beating closed models.**

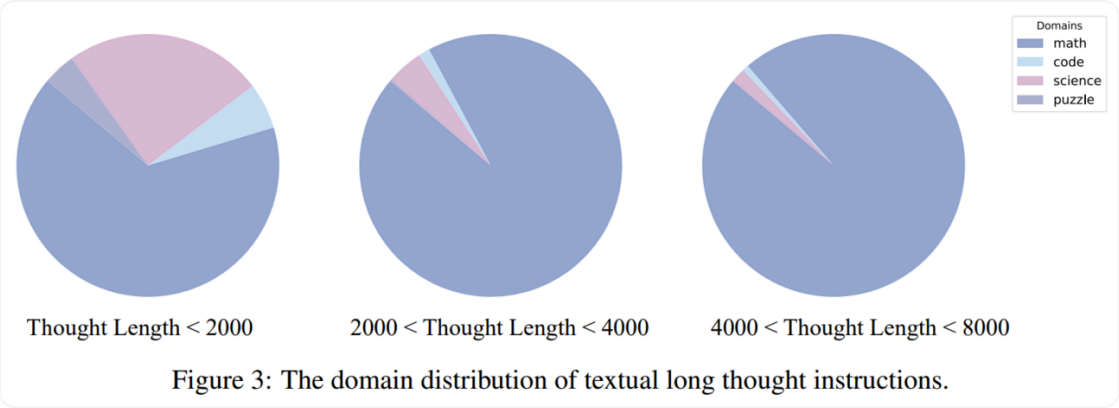

Figure 3: The domain distribution of textual long thought instructions.

Image Credit: The original "Virgo: A Preliminary Exploration on Reproducing o1-like MLLM" paper

The researchers also tried training MLLM with multimodal (image + text) reasoning data, but this did not significantly outperform the text-based training. Why? This happens because many visual problems rely more on recognizing objects or reading graphs, rather than deep reasoning.

Their experimental process results in a new system called **Virgo (Visual Reasoning with Long Thought), which enhances slow-thinking in multimodal models:**

- It scored 38.4% on MathVision and 29.3% on OlympiadBench.
- The model performed best on difficult problems that required step-by-step reasoning. For example, Virgo outperformed QVQ on hard

questions, with a 54.7% accuracy compared to QVQ's 48.6%.

Image Credit: The original "Virgo: A Preliminary Exploration on Reproducing o1-like MLLM" paper

However, **Virgo met some issues:**

- On easy and medium questions, it didn't perform as well, possibly because forcing long reasoning on simple problems makes them harder to solve.
- Also, on MMMU, the model didn't improve as much. This is likely because MMMU has simpler questions that don't require deep reasoning.
- Interestingly, the smaller Virgo-7B model performed better after multimodal training. Researchers suggested that it's because visual data better aids smaller models that struggle with complex reasoning.

Next approach introduces an upgraded way of using Monte Carlo Tree Search for increasing test-time compute for MLLMs.

Collective learning with Collective Monte Carlo Tree Search

Another interesting idea on how to teach MLLMs to think step-by-step came from Nanyang Technological University, Tsinghua and Sun Yat-sen Universities together with Baidu Inc. As MLLMs don't naturally generate structured reasoning steps, their search gets stuck in low-quality reasoning loops. To solve this, the researchers proposed to use a new **Collective Monte Carlo Tree Search (CoMCTS) method. It leverages collective learning, combining the strengths of multiple models instead of relying on just one.**

Here's how CoMCTS works step-by-step:

- **Expansion:** The AI generates multiple possible next steps instead of just one.
- **Simulation & error checking:** It tests different reasoning paths and identifies incorrect steps.
- **Backpropagation:** The model learns from mistakes and adjusts its reasoning.

- **Selection:** It chooses the best reasoning path to continue improving accuracy.

What benefits does CoMCTS offer?

- By using multiple AI models together, CoMCTS avoids getting stuck in bad reasoning loops and significantly speeds up the search for correct answers.
- **Reflective reasoning:** Instead of just finding the right answer, the MLLM learns from both correct and incorrect steps, comparing them, understands and corrects its mistakes, leading to more accurate and self-aware reasoning.

Image Credit: "Mulberry: Empowering MLLM with o1-like Reasoning and Reflection via Collective Monte Carlo Tree Search" paper

Using CoMCTS, researchers created **Mulberry-260K**, a dataset with 260,000 multimodal questions with step-by-step solutions, and common mistakes with corrections (reflective reasoning paths). Their **Mulberry MLLM** trained on this dataset showed significant improvement in multimodal reasoning:

- Mulberry-7B improved performance by 4.2% over Qwen2-VL-7B.
- Mulberry-11B improved by 7.5% over LLaMA-3.2-11B-Vision-Instruct.
- It performed better than most open-source AI models and competed with top closed-source AI models.
- It got +5.7% accuracy on MathVista (math reasoning benchmark) and +3.0% on MMMU.

Image Credit: "Mulberry: Empowering MLLM with o1-like Reasoning and Reflection via Collective Monte Carlo Tree Search" paper

Overall, this advancement proves that multimodal reasoning can be performed deeply and step-by-step, and MLLMs can reflect on their mistakes.

Generating images with CoT

A group of researchers from CUHK, MiuLar Lab, MMLab, Peking University, and Shanghai AI Lab explored another way of enhancing image generation with step-by-step reasoning. They proposed two new reward

models for **test-time verification**, called **PARM and PARM++**, to enhance image quality.

1. **Potential Assessment Reward Model (PARM)** evaluates intermediate steps and prevents poor-quality outputs through its key features:

- Clarity judgment: Determines if an image is clear enough for evaluation.
- Potential assessment: Predicts if an image can lead to a high-quality final result.
- Best-of-N' selection: Selects the best generation path based on previous judgments.

  It outperforms traditional Outcome Reward Model (ORM), which evaluates only the final image and selects the best one, **and Process Reward Model (PRM)**, that evaluates the intermediate generation steps, **by 6%, making it the best test-time verification model.**

Image Credit: "Can We Generate Images with CoT? Let's Verify and Reinforce Image Generation Step by Step" paper

2. **Potential Assessment Reward Model ++ (PARM++)** goes further and enhances PARM with a **Reflection Mechanism**, enabling self-correction of images. Here's how it works:

- It evaluates whether the final generated image aligns with the text prompt.
- If misalignment is detected, it provides detailed feedback.
- The image is iteratively refined until it meets the requirements.

PARM++ enhances image generation performance by up to 24%, surpassing Stable Diffusion 3 by 15%. It suggests an effective paradigm where AI-generated images can be continuously improved, just like human artists refine their work. Further these techniques will be applied to video generation.

That's all for now about the test-time compute paradigm in multimodality. For now, we'll dive into what can enhance already existing reasoning models in terms of their test-time compute even more, and, of course, its RAG!

Search-o1: Enhancing retrieval and agentic capabilities

The Search-o1 framework from Renmin University of China and Tsinghua University integrates large reasoning models (LRMs) like OpenAI's o1 (meaning that it may be all models that we've mentioned before) with agentic search workflow. Search-o1 helps these models reason better by allowing them to search for external knowledge when needed and filter out unnecessary information.

Here's how Search-o1 works and how it affects test-time compute:

Image Credit: "Search-o1: Agentic Search-Enhanced Large Reasoning Models" paper

- Unlike traditional models that generate answers directly, **Search-o1 pauses reasoning when it detects missing knowledge**.
- It then constructs search queries, fetches documents, and integrates the retrieved information before continuing. **Retrieving multiple documents per query increases the number of operations performed during inference.**
- If the model needs many search steps within a single reasoning task, the overall compute cost scales up significantly.

To not overload the model with too much information and retrieve only relevant and redundant information from the raw documents, Search-o1 implements **The Reason-in-Documents module**. It analyzes, summarizes, and filters the retrieved data before adding it to the reasoning chain. But, these is an issue. This extra step adds computational overhead, as it involves additional model inference to extract and condense useful information.

With all the steps which increase test-time compute Search-o1 also provides an optimization to reduce overhead in large-scale inference. It groups multiple reasoning tasks into batches. This **batch inference mechanism** enables:

- Parallel token generation for multiple reasoning tasks.
- Simultaneous retrieval of knowledge for multiple queries.
- Refinement of multiple documents at once.

This reduces redundant compute costs when handling multiple test cases.

Implementing the Search-o1 method results in significant enhancing of reasoning capabilities across various benchmarks. Just take a look at the performance results:

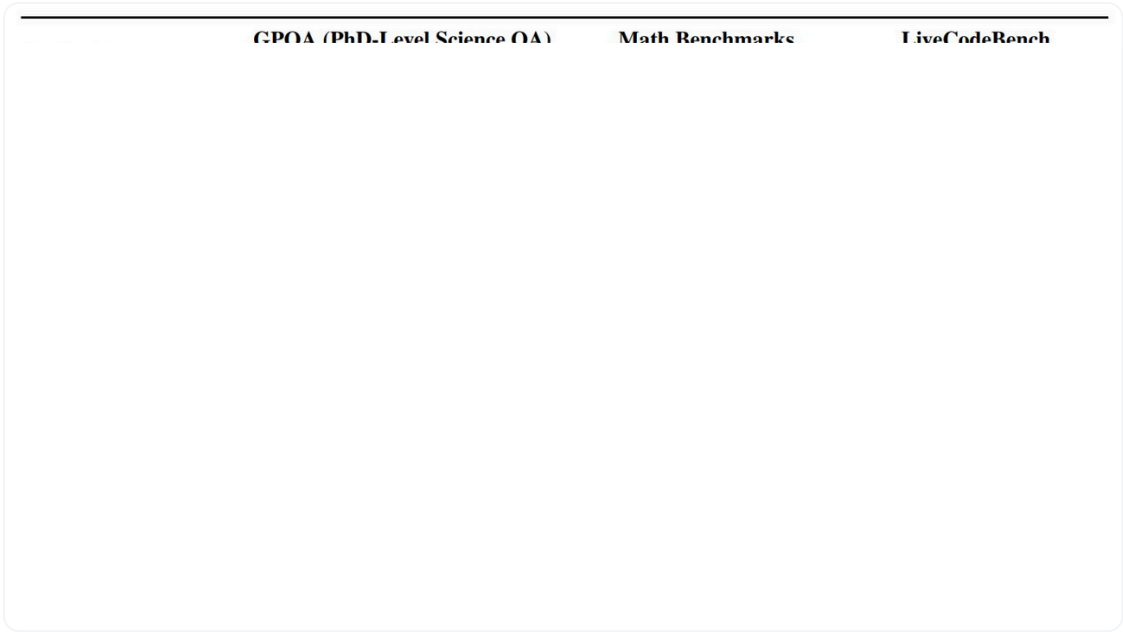

Image Credit: "Search-o1: Agentic Search-Enhanced Large Reasoning Models" paper

In this article we explored a lot of advancements in test-time compute scaling but there are three more approaches that also deserve some attention.

A brief overview of 3 more research

1. Recently, **NVIDIA** used an inference-time scaling approach in their **SANA-1.5** diffusion transformer to create more accurate images. Instead

of increasing the model size they **increased the number** of images that SANA-1.5 generates for the output. An AI "judge" (NVILA-2B model) picks the best images, based on ranking in a tournament-style selection.

2. **Stanford** proposed a simple way for test-time scaling. For training, they use **a small s1K dataset with 1,000 tough and diverse questions** with detailed reasoning steps. For effective reasoning, they leverage a special technique, **budget forcing**, which controls how long the model thinks and uses “Wait” and “Final Answer” tags to adjust the model’s reasoning time.

3. **O1-CODER** from **Beijing Jiaotong University** shifts toward System-2 thinking in coding tasks through RL and Monte Carlo Tree Search (MCTS). The model first generates pseudocode, then full code, using a Test Case Generator (TCG) for validation.

Not without limitations

Test-time compute scaling approach, even demonstrating the highest ever performance results, has its limitations that we need to mitigate further. Here they are:

- **Underthinking issue:** o1-like models may jump too quickly between different ideas, abandoning promising ones too soon.
- **Latency variability:** Responses may have inconsistent latency. Simpler queries will be fast, while complex ones may take longer, which can be problematic in real-time applications.
- **Potential over/under allocation:** Some queries might get more compute than necessary, leading to inefficiencies, while others may receive less, leading to suboptimal answers. For example, TTC scaling isn't a good option for simple questions.
- **Loss of determinism:** The same query might receive different levels of computation on different cases due to external factors like system load or model heuristics, which may lead to inconsistent outputs.
- **Unpredictable costs:** The cost per query varies, making budgeting challenging for users with highly variable queries.

Conclusion: What does the future hold for test-time compute?

Today we explored the core idea of test-time compute and five different (open-source!) methods to scale it: 1) DeepSeek-R1 reinforcement learning and cold data use; 2) approaches to test-time scale in multimodal models (using training on long-form text data, CoMCTS, and using advanced test-time verifiers); 3) Search-o1 framework which implements RAG capabilities.

Obviously, we'll meet a great amount of new research that explores methods for effective test-time scaling. It's amazing that we have an option to choose between the models that reason fast and the models that take more time for an accurate "thoughtful" reasoning process. One thing that we noticed is that slow thinking models are closer to how humans think, and though can be a more likely path to human-level intelligence.

Here is one more interesting aspect that could contribute to the further improvement of the model's reasoning during inference is test-time training.

Aravind Srinivas twitter

Test-time training (TTT) is a machine learning technique where a model continues to learn and adapt even during the test phase, rather than just making predictions with a fixed set of learned weights. It allows the model to fine-tune itself on the test data before making predictions, improving its

accuracy. Such adaptability enhances the models' ability to handle unforeseen scenarios and distribution shifts. Maybe test-time training will be the next step of enhancing reasoning models?

Author: Alyona Vert Editor: Ksenia Se

Bonus: Resources to dive deeper

- OpenAI o1 System Card by OpenAI
- DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning by DeepSeek
- Virgo: A Preliminary Exploration on Reproducing o1-like MLLM by [BAAI](Baichuan AI.), Gaoling School of Artificial Intelligence, Renmin University of China and Baichuan AI
- Mulberry: Empowering MLLM with o1-like Reasoning and Reflection via Collective Monte Carlo Tree Search by Nanyang Technological University, Tsinghua University, Baidu Inc and Sun Yat-sen University
- Can We Generate Images with CoT? Let’s Verify and Reinforce Image Generation Step by Step by CUHK MiuLar Lab & 2MMLab, Peking University, Shanghai AI Lab,

- SANA 1.5: Efficient Scaling of Training-Time and Inference-Time Compute in Linear Diffusion Transformer by NVIDIA
- s1: Simple test-time scaling by Stanford, University of Washington, Allen AI, and Contextual AI
- o1-Coder: an o1 Replication for Coding
- Thoughts Are All Over the Place: On the Underthinking of o1-Like LLMs
- Test-time Computing: from System-1 Thinking to System-2 Thinking
- O1 Replication Journey: A Strategic Progress Report -- Part 1 by NYU, Shanghai Jiao Tong University, MBZUAI and GAIR
- O1 Replication Journey -- Part 2: Surpassing O1-preview through Simple Distillation, Big Progress or Bitter Lesson?
- O1 Replication Journey -- Part 3: Inference-time Scaling for Medical Reasoning

---

📩 **If you want to receive our articles straight to your inbox, please** subscribe here

More from this author

What Coding Agent Wins?

Kseniase △ 9 June 27, 2025

🤦‍♂️ #17: What is A2A and why is it...

Kseniase △ 15 May 7, 2025

👏 Community

yanchao Feb 12, 2025

excellent work

thx

Reply

codelion Feb 17, 2025

Great survey, if you want to play around with many of the these techniques you can do so in our open-source optimizing inference proxy optillm - https://github.com/codelion/optillm

2 +

Reply

iamkaikai Feb 18, 2025

GOAT!

Reply

Edit Preview

Start discussing this article

Upload images, audio, and videos by dragging in the text input, pasting, or clicking here.

Comment · Sign up or log in to comment